Guía n°

DLX/DLQ, políticas de retención y alertas

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Subdirección de Diseño
Curricular e Instruccional
202

Guía n°

Nombre de la guía

Sigla	Asignatura	Experiencia de Aprendizaje
Tiempo	**Modalidad de Trabajo**	**Indicadores de Logro**

Contexto

- Este ejercicio te guiará paso a paso para implementar un sistema resiliente con Dead Letter Queues (DLQ), políticas de retención (TTL) y un sistema de alertas visual en tiempo real usando React.
- Objetivo: Simular un procesador de órdenes que falla, reintenta y finalmente envía los mensajes fallidos a una DLQ para análisis.

Requisitos previos

Crear docker-compose.yml

Crea un archivo en la raíz de tu proyecto:

```
version: '3.8'


services:
 rabbitmq:
   image: rabbitmq:4.2-management
   container_name: rabbitmq-dlq-demo
   ports:
```

```
    - "5672:5672"    # AMQP
    - "15672:15672"  # Management UI
  environment:
    RABBITMQ_DEFAULT_USER: guest
    RABBITMQ_DEFAULT_PASS: guest
  volumes:
    - rabbitmq_data:/var/lib/rabbitmq
  healthcheck:
    test: ["CMD", "rabbitmq-diagnostics", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5

volumes:
  rabbitmq_data:
```

Levantar RabbitMQ

```
docker-compose up -d
```

Accede al dashboard: http://localhost:15672 (usuario: guest, contraseña: guest)

Crear el Proyecto

Usa Spring Initializr (start.spring.io):

- Project: Maven
- Language: Java
- Spring Boot: 3.1.x o superior
- Dependencies: Spring Web, Spring for RabbitMQ
- Group: com.example
- Artifact: order-processor-dlq

Paso 2.2: Configurar RabbitMQ con DLX/DLQ

Crea el archivo src/main/java/com/example/RabbitMQConfig.java:

```
package com.example;

import org.springframework.amqp.core.*;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitMQConfig {
// ========== COLAS Y EXCHANGES PRINCIPALES ==========
    public static final String ORDERS_EXCHANGE = "orders.exchange";
    public static final String ORDERS_QUEUE = "orders.queue";
    public static final String ORDERS_ROUTING_KEY = "order.created";

    // ========== DLX (Dead Letter Exchange) ==========
    public static final String DLX_EXCHANGE = "orders.dlx";
    public static final String DLQ_QUEUE = "orders.dlq";
    public static final String DLX_ROUTING_KEY = "order.dead";

    // ========== Exchange Principal (Direct) ==========
```

```
    @Bean
    public DirectExchange ordersExchange() {
        return new DirectExchange(ORDERS_EXCHANGE, true, false);
    }

    // ========== COLA PRINCIPAL CON DLX CONFIGURADO ==========
    /**
     * La magia está aquí. Esta cola:
     * 1. Es durable (sobrevive reinicio de RabbitMQ)
     * 2. Tiene TTL de 30 segundos para los mensajes
     * 3. Está configurada para enviar mensajes muertos al DLX
     * 4. Reintentará con exponential backoff (lo configura Spring)
     */
    @Bean
    public Queue ordersQueue() {
        return QueueBuilder.durable(ORDERS_QUEUE)
            // TTL: Los mensajes expiran después de 30 segundos
            .withArgument("x-message-ttl", 30000)
            // DLX: Si el mensaje falla, va aquí
            .withArgument("x-dead-letter-exchange", DLX_EXCHANGE)
            .withArgument("x-dead-letter-routing-key", DLX_ROUTING_KEY)
            // Max retries a nivel de cola (opcional)
            .withArgument("x-max-length", 1000) // Máximo 1000 mensajes
            .build();
    }

    @Bean
    public Binding ordersBinding(Queue ordersQueue, DirectExchange
ordersExchange) {
        return BindingBuilder.bind(ordersQueue)
            .to(ordersExchange)
            .with(ORDERS_ROUTING_KEY);
    }

    // ========== DEAD LETTER EXCHANGE (DLX) Y COLA (DLQ) ==========
```

```
    /**
     * El DLX recibe todos los mensajes que fallaron en la cola principal.
     * Usamos Fanout para que todos los listeners lo vean.
     */
    @Bean
    public FanoutExchange deadLetterExchange() {
        return new FanoutExchange(DLX_EXCHANGE, true, false);
    }

    /**
     * La DLQ guarda los mensajes fallidos para auditoría y análisis.
     * TTL de 24 horas para mantener un histórico.
     */
    @Bean
    public Queue deadLetterQueue() {
        return QueueBuilder.durable(DLQ_QUEUE)
            // TTL: Los mensajes en DLQ se borran después de 24h
            .withArgument("x-message-ttl", 86400000) // 24 horas en ms
            .build();
    }

    @Bean
    public Binding deadLetterBinding(Queue deadLetterQueue,
FanoutExchange deadLetterExchange) {
        return BindingBuilder.bind(deadLetterQueue)
            .to(deadLetterExchange);
    }
}
```

Paso 2.3: Configurar Reintentos en application.yml

```
 rabbitmq:
   host: localhost
```

```
    port: 5672
    username: guest
    password: guest

    listener:
      simple:
        # Modo manual: El consumidor decide cuándo hacer Ack
        acknowledge-mode: manual

        # Prefetch: Procesa 1 mensaje a la vez (Fair Dispatch)
        prefetch: 1

        # Reintentos con Exponential Backoff
        retry:
          enabled: true
          initial-interval: 1000      # 1 segundo
          max-interval: 10000         # 10 segundos máximo
          multiplier: 2.0           # Duplica cada intento
          max-attempts: 3           # 3 intentos totales

server:
  port: 8080
```

Paso 2.4: Crear el Consumidor

src/main/java/com/example/OrderConsumer.java:

```
package com.example;

import org.springframework.amqp.rabbit.annotation.RabbitListener;
```

```
import org.springframework.amqp.support.AmqpHeaders;
import org.springframework.messaging.handler.annotation.Header;
import org.springframework.stereotype.Service;
import com.rabbitmq.client.Channel;

@Service
public class OrderConsumer {

    /**
     * Consumidor principal: Recibe órdenes de la cola principal.
     * Simula fallos aleatorios para demostrar el DLX.
     */
    @RabbitListener(queues = RabbitMQConfig.ORDERS_QUEUE)
    public void processOrder(
            String message,
            Channel channel,
            @Header(AmqpHeaders.DELIVERY_TAG) long deliveryTag) {

        try {
            System.out.println("[PROCESADOR] Recibida orden: " + message);

            // SIMULACIÓN DE FALLO ALEATORIO
            // 50% de probabilidad de fallar en los primeros 2 intentos
            if (Math.random() < 0.5) {
                throw new RuntimeException(" Error simulado al procesar: " +
message);
            }

            // Si llegamos aquí, procesamiento exitoso
            System.out.println("[✓ ÉXITO] Orden procesada correctamente: " +
message);

            // Ack manual: Le decimos a RabbitMQ que todo estuvo bien
            channel.basicAck(deliveryTag, false);
```

```
        } catch (Exception e) {
            System.out.println("[⚠️ ERROR] " + e.getMessage());

            try {
                // Nack con requeue=false: Va al DLX, no se reintentar aquí
                channel.basicNack(deliveryTag, false, false);
                System.out.println("[→ DLX] Mensaje enviado a Dead Letter
Exchange");

            } catch (Exception nackException) {
                nackException.printStackTrace();
            }
        }
    }

    /**
     * Consumidor de DLQ: Monitorea los mensajes que fallaron
definitivamente.
     */
    @RabbitListener(queues = RabbitMQConfig.DLQ_QUEUE)
    public void processDLQ(String message) {
        System.out.println(" [DLQ] MENSAJE EN CUARENTENA: " + message);
        System.out.println("    → Revisar logs del procesador para diagrama del
fallo.");
    }
}
```

Paso 2.5: Crear el Controlador REST

src/main/java/com/example/OrderController.java:

```
package com.example;

import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.web.bind.annotation.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/api/orders")
@CrossOrigin(origins = "http://localhost:5173")
public class OrderController {

    @Autowired
    private RabbitTemplate rabbitTemplate;

    @PostMapping("/send")
    public Map<String, String> sendOrder(@RequestBody Map<String, String>
payload) {
        String orderId = payload.getOrDefault("orderId", "ORD-" +
System.currentTimeMillis());
        String customerName = payload.getOrDefault("customerName",
"Unknown");

        String message = String.format(
            "Orden ID: %s | Cliente: %s | Hora: %s",
            orderId,
            customerName,

LocalDateTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss"))
        );

        // Enviar al Exchange (el routing automático se encarga del resto)
        rabbitTemplate.convertAndSend(
            RabbitMQConfig.ORDERS_EXCHANGE,
            RabbitMQConfig.ORDERS_ROUTING_KEY,
            message
        );
```

```
        Map<String, String> response = new HashMap<>();
        response.put("status", "Orden enviada");
        response.put("orderId", orderId);
        response.put("message", message);
        return response;
    }

    @GetMapping("/status")
    public Map<String, String> getStatus() {
        Map<String, String> status = new HashMap<>();
        status.put("backend", "Online");
        status.put("rabbitmq", "Conectado");
        status.put("timestamp", LocalDateTime.now().toString());
        return status;
    }
}
```

Frontend Vite + React

Paso 3.1: Crear el Proyecto

```
npm create vite@latest -- --template react
cd <project-name>
npm install
```

Paso 3.2: Crear el Componente Principal

Reemplaza src/App.jsx:

```
import React, { useState, useEffect } from 'react';
import './App.css';function App() {
 const [orders, setOrders] = useState([]);
```

```
const [status, setStatus] = useState('Conectando...');
const [stats, setStats] = useState({ total: 0, success: 0, failed: 0 });

// Cargar estado inicial
useEffect(() => {
  checkBackendStatus();
  const interval = setInterval(checkBackendStatus, 5000);
  return () => clearInterval(interval);
}, []);

const checkBackendStatus = async () => {
  try {
    const response = await fetch('http://localhost:8080/api/orders/status');
    if (response.ok) {
      setStatus('✓ Conectado a Backend');
    }
  } catch {
    setStatus('✗ Backend desconectado');
  }
};

const sendOrder = async (customerName) => {
  const orderId = `ORD-${Date.now()}`;

  try {
    const response = await fetch('http://localhost:8080/api/orders/send', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ orderId, customerName }),
    });

    if (response.ok) {
      const data = await response.json();

      // Agregar a la lista de órdenes
      const newOrder = {
```

```
        id: orderId,
        customer: customerName,
        message: data.message,
        timestamp: new Date().toLocaleTimeString(),
        status: 'ENVIADA', // Pendiente confirmación del servidor
      };

      setOrders([newOrder, ...orders]);
      setStats(prev => ({ ...prev, total: prev.total + 1 }));

      // Simular resultado después de 2 segundos (en realidad verías en los
logs)
      setTimeout(() => {
        updateOrderStatus(orderId);
      }, 2000);
    }
  } catch (error) {
    console.error('Error:', error);
    alert('Error al enviar la orden');
  }
};

const updateOrderStatus = (orderId) => {
  // En una app real, esto vendría de un WebSocket
  // Por ahora, simulamos que hay 60% de éxito
  const isSuccess = Math.random() > 0.4;

  setOrders(prevOrders =>
    prevOrders.map(order =>
      order.id === orderId
        ? {
            ...order,
            status: isSuccess ? '✓ PROCESADA' : '✗ FALLIDA (DLQ)',
          }
        : order
```

```
    )
  );

  if (isSuccess) {
    setStats(prev => ({ ...prev, success: prev.success + 1 }));
  } else {
    setStats(prev => ({ ...prev, failed: prev.failed + 1 }));
  }
 };

 return (
  <div className="App">
    <header className="header">
      <h1> Sistema de Órdenes con RabbitMQ</h1>
      <p className="status-badge">{status}</p>
    </header>

    <section className="control-panel">
      <h2>Enviar Órdenes</h2>
      <div className="button-group">
        <button onClick={() => sendOrder('Juan Pérez')} className="btn btn-
primary">
          Orden Cliente 1
        </button>
        <button onClick={() => sendOrder('María García')} className="btn btn-
primary">
          Orden Cliente 2
        </button>
        <button onClick={() => sendOrder('Carlos López')} className="btn btn-
primary">
          Orden Cliente 3
        </button>
      </div>
    </section>

    <section className="stats-panel">
```

```
  <div className="stat-card">
   <div className="stat-number">{stats.total}</div>
   <div className="stat-label">Total Órdenes</div>
  </div>
  <div className="stat-card success">
   <div className="stat-number">{stats.success}</div>
   <div className="stat-label">Procesadas</div>
  </div>
  <div className="stat-card error">
   <div className="stat-number">{stats.failed}</div>
   <div className="stat-label">En DLQ</div>
  </div>
 </section>

 <section className="orders-list">
  <h2> Historial de Órdenes</h2>
  {orders.length === 0 ? (
   <p className="empty-message">No hay órdenes aún. ¡Envía una!</p>
  ) : (
   <div className="orders-table">
    {orders.map(order => (
     <div key={order.id} className={`order-item status-${order.status}`}>
      <div className="order-header">
       <strong>{order.id}</strong>
       <span className={`status-badge status-${order.status}`}>
        {order.status}
       </span>
      </div>
      <div className="order-detail">
       <span>Cliente: {order.customer}</span>
       <span className="timestamp">{order.timestamp}</span>
      </div>
      <div className="order-message">{order.message}</div>
     </div>
    ))}
```

```
        </div>
       )}
      </section>

      <footer className="footer">
        <p> Tip: Abre la consola de RabbitMQ (localhost:15672) para ver las
colas en tiempo real</p>
      </footer>
    </div>
  );
}

export default App;
```

Paso 3.3: Agregar Estilos

Reemplaza src/App.css:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  min-height: 100vh;
}

.App {
  max-width: 1200px;
  margin: 0 auto;
  padding: 2rem;
}
```

```
.header {
  background: white;
  border-radius: 12px;
  padding: 2rem;
  margin-bottom: 2rem;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  text-align: center;
}

.header h1 {
  color: #333;
  margin-bottom: 1rem;
}

.status-badge {
  display: inline-block;
  padding: 0.5rem 1.5rem;
  background: #10b981;
  color: white;
  border-radius: 20px;
  font-weight: 600;
  font-size: 0.9rem;
}

.control-panel {
  background: white;
  border-radius: 12px;
  padding: 2rem;
  margin-bottom: 2rem;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

.control-panel h2 {
  margin-bottom: 1.5rem;
  color: #333;
}
```

```
.button-group {
  display: flex;
  gap: 1rem;
  flex-wrap: wrap;
}

.btn {
  padding: 0.75rem 1.5rem;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  font-size: 1rem;
  font-weight: 600;
  transition: all 0.3s ease;
}

.btn-primary {
  background: #3b82f6;
  color: white;
}

.btn-primary:hover {
  background: #2563eb;
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(59, 130, 246, 0.4);
}

.stats-panel {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 1.5rem;
  margin-bottom: 2rem;
}

.stat-card {
```

```
  background: white;
  border-radius: 12px;
  padding: 1.5rem;
  text-align: center;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  border-left: 4px solid #3b82f6;
}

.stat-card.success {
  border-left-color: #10b981;
}

.stat-card.error {
  border-left-color: #ef4444;
}

.stat-number {
  font-size: 2.5rem;
  font-weight: 700;
  color: #333;
  margin-bottom: 0.5rem;
}

.stat-label {
  color: #666;
  font-size: 0.95rem;
}

.orders-list {  background: white;
  border-radius: 12px;
  padding: 2rem;
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

.orders-list h2 {
  margin-bottom: 1.5rem;
```

```
  color: #333;
}

.orders-table {
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.order-item {
  border-left: 4px solid #3b82f6;
  padding: 1rem;
  border-radius: 8px;
  background: #f9fafb;
  transition: all 0.3s ease;
}

.order-item.status-PROCESADA {
  border-left-color: #10b981;
  background: #ecfdf5;
}

.order-item.status-FALLIDA {
  border-left-color: #ef4444;
  background: #fef2f2;
}

.order-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 0.5rem;
}

.order-detail {
  display: flex;
```

```
  justify-content: space-between;
  font-size: 0.9rem;
  color: #666;
  margin-bottom: 0.5rem;
}

.timestamp {
  color: #999;
  font-size: 0.85rem;
}

.order-message {
  color: #333;
  padding: 0.5rem 0;
  font-size: 0.95rem;
}

.empty-message {
  text-align: center;
  color: #999;
  padding: 2rem;
}

.footer {
  text-align: center;
  color: white;
  margin-top: 2rem;
  font-size: 0.9rem;
}
```

Parte 4: Prueba Completa

Paso 4.1: Inicia todo

Terminal 1 - Backend:

```
cd order-processor-dlq
mvn spring-boot:run
```

Terminal 2 - Frontend:

```
cd vite-order-app
npm run dev
```

Terminal 3 - Monitoreo (Opcional):

```
docker logs -f rabbitmq-dlq-demo
```

Paso 4.2: Realiza pruebas

1. Abre http://localhost:5173 en tu navegador.
2. Haz clic en "Orden Cliente 1".
3. Observa los logs del backend:
    a. Verás si procesó correctamente o falló.
    b. Si falla, verás el mensaje "→ DLX".
4. Abre http://localhost:15672 (RabbitMQ UI).
5. Ve a "Queues" y observa:
    a. orders.queue: Las órdenes siendo procesadas.
    b. orders.dlq: Los mensajes que fallaron (visible después de 3 intentos).

Paso 4.3: Ver en el Dashboard

En RabbitMQ Management:

- Ve a "Queues"

- Selecciona orders.queue
- Verás métricas en tiempo real: mensajes entrantes, salientes, y en DLQ.

Conceptos Implementados

Concepto	Dónde aparece	Por qué
DLX/DLQ	RabbitMQConfig (x-dead-letter-exchange)	Captura automáticamente mensajes fallidos
TTL	RabbitMQConfig (x-message-ttl)	Los mensajes expiran después de 30s (en DLQ, 24h)
Reintentos	application.yml (retry.enabled: true)	3 intentos con backoff exponencial (1s, 2s, 4s)
Ack Manual	OrderConsumer (basicAck / basicNack)	Control total de cuándo confirmar
Prefetch	application.yml (prefetch: 1)	Fair dispatch: 1 mensaje por worker

Links complementarios

1. RabbitMQ Core (Documentación Oficial del Broker)

Estos enlaces explican cómo funciona la lógica en el servidor RabbitMQ, independientemente del lenguaje de programación.

- Dead Letter Exchanges (DLX): Explica los argumentos x-dead-letter-exchange y por qué un mensaje acaba ahí (reject, expire, maxlen).
  - https://www.rabbitmq.com/docs/dlx
- Time-To-Live (TTL): Detalla cómo funcionan x-message-ttl (por mensaje) y x-expires (por cola).
  - https://www.rabbitmq.com/docs/ttl

2. Spring Boot & Spring AMQP (Librería Java)

Estos enlaces son específicos para la implementación en código Java que hicimos.

- Configuring the Broker (QueueBuilder): Documentación sobre cómo usar QueueBuilder.durable().withArgument(...) para configurar DLQs y TTLs programáticamente.
  - https://docs.spring.io/spring-amqp/reference/amqp/broker-configuration.html
- Retry Support (Backoff): Explica cómo configurar spring.rabbitmq.listener.simple.retry en el application.yml para los reintentos automáticos.
  - https://docs.spring.io/spring-amqp/reference/amqp/receiving-messages/async-consumer.html#container-retry
- Message Listener Container: Detalles profundos sobre el SimpleMessageListenerContainer, que es el motor que procesa los mensajes y maneja los Acks manuales.
  - https://docs.spring.io/spring-amqp/reference/amqp/receiving-messages/async-consumer.html

3. React & Docker (Frontend e Infra)

- Docker Hub - RabbitMQ Official Image: Documentación de la imagen rabbitmq:management que usamos en el docker-compose.yml.
  - https://hub.docker.com/_/rabbitmq
- Vite Documentation: Guía oficial para el setup de React.
  - https://vitejs.dev/guide/